

MACHINE LEARNING — LAB MANUAL

Lab 4: Improving and Comparing Machine Learning Models

A 3-Hour Hands-On Lab — BS Computer Science, 5th Semester

Duration: 3 Hours (independent, hands-on)

Prerequisites: Lab 1 (Python/tools, Kaggle & Colab); Lab 2 (cleaning & EDA); Lab 3 (Decision Tree baseline, evaluation, overfitting) — all on the Telco Churn dataset

Tools Required: Python 3.x, Pandas, NumPy, scikit-learn, Matplotlib, Google Colab

Includes: Random Forest, cross-validation, hyperparameter tuning, model comparison, feature importance, a Kaggle-style submission, and a graded rubric

Learning Objectives

This lab is NOT a re-introduction to Python, Pandas, EDA, or Decision Tree theory — you already know these from Labs 1–3. The goal is to practice improving and rigorously comparing models on the same problem you've been working on since Lab 2. By the end of this lab you should be able to independently:

- Train and evaluate a Random Forest classifier as an ensemble alternative to a single Decision Tree
- Compare two models fairly using accuracy, precision, recall, F1-score, and a confusion matrix
- Use k-fold cross-validation to get a more reliable estimate of model performance
- Perform basic hyperparameter tuning with GridSearchCV or RandomizedSearchCV
- Compare baseline vs. tuned models and justify which is genuinely better
- Read feature importances from an ensemble model and interpret them
- Select and justify a final model based on evidence, not convenience
- Prepare a Kaggle-style prediction/submission file from a trained model

Module	Focus Area
1	Decision Tree Baseline Review (0–20 min)
2	Random Forest (20–55 min)
3	Evaluation & Comparison (55–85 min)
4	Cross-Validation (85–120 min)
5	Hyperparameter Tuning (120–150 min)
6	Final Comparison & Feature Importance (150–170 min)
7	Kaggle Submission & Conclusion (170–180 min)

Dataset & Problem

You continue with the same Telco Customer Churn dataset and problem from Labs 2–3: predicting whether a customer will churn (Yes/No). Today's job is to move beyond a single Decision Tree — build a stronger ensemble model, evaluate it rigorously with cross-validation, tune it, and decide, with evidence, which model you would actually deploy.

Before You Start

- Open your Lab 3 notebook and locate your cleaned features (X, y) and your Decision Tree baseline results.
- Create a new Colab notebook: lab4_<yourname>_churn_models.ipynb, with the student-info block and Markdown headings described in Submission Requirements.
- You will submit only this one notebook — make sure every section asked for is present before you finish.

Part 1 — Decision Tree Baseline Review

Before improving anything, re-establish your Lab 3 Decision Tree result as the benchmark everything else in this lab is measured against.

Concepts / Functions You May Find Useful

Concept / Function	Purpose
X, y, X_train, X_test, y_train, y_test	Re-load or re-create your Lab 3 features/split
DecisionTreeClassifier(max_depth=...)	Retrain your Lab 3 chosen-depth tree
accuracy_score, precision_score, recall_score, f1_score	Recompute its test-set metrics

Tasks

1. Re-load your cleaned Lab 2 dataset and recreate the same X/y and train/test split you used in Lab 3 (same random_state).
2. Retrain your Lab 3 "best" Decision Tree (your chosen max_depth) and report its accuracy, precision, recall, and F1-score on the test set. This is your benchmark for the rest of the lab.

Part 2 — Random Forest

A Random Forest trains many Decision Trees on bootstrapped samples and averages their predictions. You already understand individual trees — focus here on how the ensemble differs in behavior, not on re-learning tree theory.

Concepts / Functions You May Find Useful

Concept / Function	Purpose
from sklearn.ensemble import RandomForestClassifier	Import the ensemble classifier
RandomForestClassifier(n_estimators=..., random_state=...)	Creates a Random Forest with a given number of trees
.fit(X_train, y_train) / .predict(X)	Train and predict, exactly like a Decision Tree

Tasks

3. Train a RandomForestClassifier on the same training data (reasonable default n_estimators, e.g. 100–300; fix random_state). Generate predictions on the test set.
4. In 1–2 sentences, explain in your own words why a Random Forest might generalize better than a single unrestricted Decision Tree.

Part 3 — Evaluation & Comparison

Evaluate the Random Forest the same rigorous way you evaluated the Decision Tree in Lab 3, then place both models side by side.

Concepts / Functions You May Find Useful

Concept / Function	Purpose
confusion_matrix / ConfusionMatrixDisplay	Visualize correct vs. incorrect predictions by class
classification_report(y_test, y_pred)	Summarizes precision/recall/F1 per class

Tasks

5. Compute accuracy, precision, recall, and F1-score for the Random Forest on the test set, and plot its confusion matrix.
6. Build a single comparison table: Decision Tree (Part 1) vs. Random Forest (Part 2), across all four metrics.
7. Which model performs better, and on which specific metric(s)? Is the difference large or marginal? Given Lab 2/3's finding about class imbalance, which metric matters most here?

Part 4 — Cross-Validation

A single train/test split gives one noisy estimate of performance. k-fold cross-validation trains and tests the model k times on different splits, giving a more reliable picture.

Concepts / Functions You May Find Useful

Concept / Function	Purpose
from sklearn.model_selection import cross_val_score	Runs k-fold cross-validation for a given model/metric
cross_val_score(model, X, y, cv=5, scoring='f1')	Returns one score per fold
.mean() / .std()	Summarize the fold scores

Tasks

8. Run 5-fold cross-validation for both the Decision Tree and the Random Forest, using a metric other than plain accuracy (e.g. F1-score). Report the mean and standard deviation of each model's fold scores.
9. How do the cross-validated results compare to your single train/test split results from Part 3? Do they change your view of which model is better, or how confident you are in that conclusion?

Part 5 — Hyperparameter Tuning

Search over a small grid of hyperparameters to see whether a tuned model meaningfully beats your default-settings baseline. Keep the search small — this is about the workflow, not an exhaustive search.

Concepts / Functions You May Find Useful

Concept / Function	Purpose
<code>from sklearn.model_selection import GridSearchCV, RandomizedSearchCV</code>	Import the tuning tools
<code>param_grid = {'n_estimators': [...], 'max_depth': [...], ...}</code>	Defines the hyperparameter search space
<code>GridSearchCV(model, param_grid, cv=5, scoring='f1')</code>	Searches the grid using cross-validation
<code>.best_params_ / .best_score_ / .best_estimator_</code>	Inspect the winning configuration

Tasks

- Choose 2–3 Random Forest hyperparameters to tune (e.g. `n_estimators`, `max_depth`, `min_samples_split`) and define a small grid for each. Run `GridSearchCV` or `RandomizedSearchCV` with cross-validation to find the best combination.
- Report the best hyperparameters found and the corresponding cross-validated score.
- Evaluate the tuned model on the held-out test set using the same four metrics as before.

Part 6 — Final Comparison & Feature Importance

Bring every model you've built into one table, choose a final model with evidence, and look inside it to see what it actually learned.

Concepts / Functions You May Find Useful

Concept / Function	Purpose
<code>model.feature_importances_</code>	Relative importance scikit-learn assigned each feature
<code>pd.Series(importances, index=X.columns).sort_values()</code>	Rank features by importance
<code>plt.barh(...)</code>	Visualize feature importances

Tasks

- Build one final comparison table across all models you trained: Decision Tree, default Random Forest, and tuned Random Forest — accuracy, precision, recall, F1-score for each.
- Select your final model and justify the choice in 2–3 sentences using evidence from the table above — the best training score alone is not sufficient justification.
- Extract and visualize `feature_importances_` for your final model. Which 3–5 features matter most, and do they agree with what Lab 2's EDA and Lab 3's Decision Tree suggested?

Part 7 — Kaggle Submission & Conclusion

Wrap up by producing predictions in the shape a real Kaggle competition would expect, and reflect on the lab as a whole.

Tasks

16. Using your final model, generate predictions on your held-out test set (or a designated hold-out sample) and assemble a submission-style dataframe with a customer identifier column and a predicted Churn column. Save it as submission.csv.
17. Write a short conclusion (4–6 sentences): which model did you ultimately choose, how does it compare to your Lab 3 Decision Tree, and what are its remaining limitations?
18. What would you try next if you had more time (e.g. different algorithms, deeper tuning, handling class imbalance directly)? Describe it — do not implement it here.

Submission Requirements

Submit exactly ONE Google Colab notebook. At the very top, include a student-info block with: Name, Student ID, Section, GitHub Profile, Kaggle Profile, and Dataset information.

Your Notebook Must Contain These Sections

- 1. Decision Tree Baseline 2. Random Forest 3. Model Evaluation 4. Decision Tree vs Random Forest
- 5. Cross-Validation 6. Hyperparameter Tuning 7. Final Model Comparison 8. Feature Importance
- 9. Final Model Selection 10. Kaggle Prediction/Submission 11. Conclusion

Every table, plot, and metric must be accompanied by a short written interpretation — code with no discussion will not be accepted.

Penalties

- Comparing models using accuracy only (no precision/recall/F1/confusion matrix): marks deducted in Part 3.
- Skipping cross-validation, or misreading its results relative to the single-split comparison: marks deducted in Part 4.
- Hyperparameter search with no cross-validation, or an unreasonably large/slow grid: marks deducted in Part 5.
- Selecting a final model with no justification, or by training score alone: marks deducted in Part 6.
- Feature importances reported with no interpretation: no credit for that section.
- Missing the Kaggle-style submission file or the student-info block: marks deducted.